WIKIPEDIA 25
25 years of the free encyclopedia

k-means clustering

***k*-means clustering** is a method of vector quantization, originally from signal processing, that aims to partition *n* observations into *k* clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or cluster centroid). This results in a partitioning of the data space into Voronoi cells. *k*-means clustering minimizes within-cluster variances (squared Euclidean distances), but not regular Euclidean distances, which would be the more difficult Weber problem: the mean optimizes squared errors, whereas only the geometric median minimizes Euclidean distances. For instance, better Euclidean solutions can be found using *k*-medians and *k*-medoids.

The problem is computationally difficult (NP-hard); however, efficient heuristic algorithms converge quickly to a local optimum. These are usually similar to the expectation–maximization algorithm for mixtures of Gaussian distributions via an iterative refinement approach employed by both *k-means* and *Gaussian mixture modeling*. They both use cluster centers to model the data; however, *k*-means clustering tends to find clusters of comparable spatial extent, while the Gaussian mixture model allows clusters to have different shapes.

The unsupervised *k*-means algorithm has a loose relationship to the *k*-nearest neighbor classifier, a popular supervised machine learning technique for classification that is often confused with *k*-means due to the name. Applying the 1-nearest neighbor classifier to the cluster centers obtained by *k*-means classifies new data into the existing clusters. This is known as nearest centroid classifier or Rocchio algorithm.

Description

Given a set of observations $(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n)$, where each observation is a d-dimensional real vector, *k*-means clustering aims to partition the n observations into k ($\leq n$) sets $\mathbf{S} = \{S_1, S_2, \ldots, S_k\}$ so as to minimize the within-cluster sum of squares (WCSS) (i.e. variance). Formally, the objective is to find:

$$\operatorname*{arg\,min}_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 = \operatorname*{arg\,min}_{\mathbf{S}} \sum_{i=1}^{k} |S_i| \operatorname{Var} S_i$$

where $\boldsymbol{\mu}_i$ is the mean (also called centroid) of points in S_i, i.e.

$$\boldsymbol{\mu}_i = \frac{1}{|S_i|} \sum_{\mathbf{x} \in S_i} \mathbf{x},$$

$|S_i|$ is the size of S_i, and $\|\cdot\|$ is the usual L^2 norm . This is equivalent to minimizing the pairwise squared deviations of points in the same cluster:

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \frac{1}{|S_i|} \sum_{\mathbf{x},\mathbf{y}\in S_i} \|\mathbf{x}-\mathbf{y}\|^2$$

The equivalence can be deduced from the identity $|S_i| \sum_{\mathbf{x}\in S_i} \|\mathbf{x}-\boldsymbol{\mu}_i\|^2 = \frac{1}{2} \sum_{\mathbf{x},\mathbf{y}\in S_i} \|\mathbf{x}-\mathbf{y}\|^2$. Since the total variance is constant, this is equivalent to maximizing the sum of squared deviations between points in *different* clusters (between-cluster sum of squares, BCSS).[1] This deterministic relationship is also related to the law of total variance in probability theory.

History

The term "*k*-means" was first used by James MacQueen in 1967,[2] though the idea goes back to Hugo Steinhaus in 1956.[3] The standard algorithm was first proposed by Stuart Lloyd of Bell Labs in 1957 as a technique for pulse-code modulation, although it was not published as a journal article until 1982.[4] In 1965, Edward W. Forgy published essentially the same method, which is why it is sometimes referred to as the Lloyd–Forgy algorithm.[5]

Early applications of the k-means algorithm were primarily found in signal processing and data compression, particularly in the context of vector quantization. Lloyd's work at Bell Labs focused on representing analog signals using a limited set of discrete values, where clustering was used to reduce the amount of data required while preserving signal quality.[4]



Aerial view of Bell Labs Holmdel Complex, where Stuart Lloyd developed early k-means clustering methods for signal processing and vector quantization

As computing power increased, k-means clustering became widely used in pattern recognition and statistical classification due to its simplicity and computational efficiency. It was later adopted in early machine learning and data analysis tasks involving large datasets.[6]

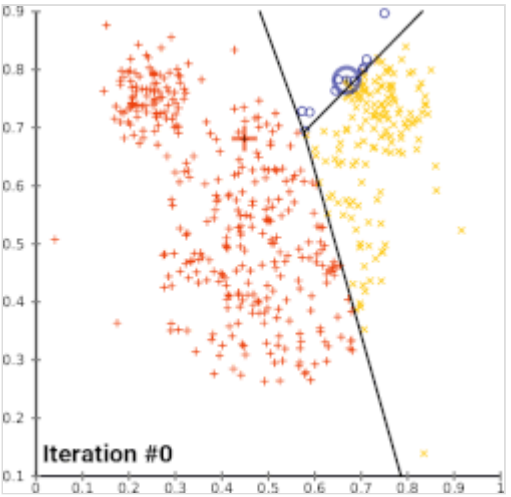
Despite its widespread use, limitations such as sensitivity to initial centroid placement and difficulty handling non-spherical clusters were recognized early on, motivating the development of improved clustering methods and initialization techniques.

Numerous extensions of k-means have since been developed to address limitations of the original algorithm, including methods such as fuzzy c-means, which allows data points to belong to multiple clusters with varying degrees of membership, and kernel k-means, which uses kernel functions to identify non-linearly separable clusters.[6]

Algorithms

Standard algorithm (naive *k*-means)

The most common algorithm uses an iterative refinement technique. Due to its ubiquity, it is often called "the *k*-means algorithm"; it is also referred to as Lloyd's algorithm, particularly in the computer science community. It is sometimes also referred to as "naïve *k*-means", because there exist much faster alternatives.[7]



Convergence of *k*-means

Given an initial set of k means $m_1^{(1)}$, ..., $m_k^{(1)}$ (see below), the algorithm proceeds by alternating between two steps:[8]

1. **Assignment step**: Assign each observation to the cluster with the nearest mean (centroid): that with the least squared Euclidean distance.[9] (Mathematically, this means partitioning the observations according to the Voronoi diagram generated by the means.)

$$S_i^{(t)} = \left\{ x_p : \left\| x_p - m_i^{(t)} \right\|^2 \leq \left\| x_p - m_j^{(t)} \right\|^2 \ \forall j, 1 \leq j \leq k \right\},$$

   where each x_p is assigned to exactly one $S^{(t)}$, even if it could be assigned to two or more of them.
2. **Update step**: Recalculate means (centroids) for observations assigned to each cluster. This is also called refitting.

$$m_i^{(t+1)} = \frac{1}{\left| S_i^{(t)} \right|} \sum_{x_j \in S_i^{(t)}} x_j$$

The objective function in *k*-means is the WCSS (within cluster sum of squares). After each iteration, the WCSS monotonically decreases, giving a nonnegative monotonically decreasing sequence. This guarantees that the *k*-means always converges, but not necessarily to the global optimum.

The algorithm has converged when the assignments no longer change or equivalently, when the WCSS has become stable. The algorithm is not guaranteed to find the optimal cluster assignment.[10]

The algorithm is often presented as assigning objects to the nearest cluster by distance. Using a different distance function other than (squared) Euclidean distance may prevent the algorithm from converging. Various modifications of *k*-means such as spherical *k*-means and *k*-medoids have been proposed to allow using other distance measures.

Pseudocode

The below pseudocode outlines the implementation of the standard *k*-means clustering algorithm. Initialization of centroids, distance metric between points and centroids, and the calculation of new centroids are design choices and will vary with different implementations. In this example pseudocode, `distance()` returns the distance between the specified points.

```
function kmeans(k, points) is
    // Initialize centroids
    centroids ← list of k starting centroids
    converged ← false

    while converged == false do
        // Create empty clusters
        clusters ← list of k empty lists

        // Assign each point to the nearest centroid
        for i ← 0 to length(points) - 1 do
            point ← points[i]
            closestIndex ← 0
            minDistance ← distance(point, centroids[0])
            for j ← 1 to k - 1 do
                d ← distance(point, centroids[j])
                if d < minDistance THEN
                    minDistance ← d
                    closestIndex ← j
            clusters[closestIndex].append(point)

        // Recalculate centroids as the mean of each cluster
        newCentroids ← empty list
        for i ← 0 to k - 1 do
            newCentroid ← calculateCentroid(clusters[i])
            newCentroids.append(newCentroid)

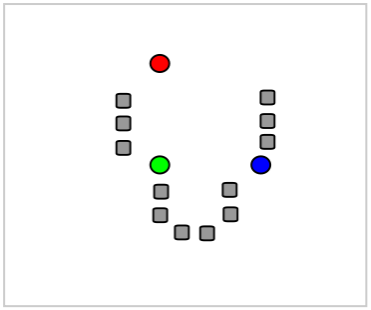
        // Check for convergence
        if newCentroids == centroids THEN
            converged ← true
        else
            centroids ← newCentroids

    return clusters
```

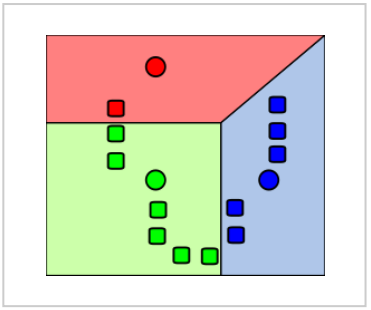
Initialization methods

Commonly used initialization methods are Forgy and Random Partition.[11] The Forgy method randomly chooses *k* observations from the dataset and uses these as the initial means. The Random Partition method first randomly assigns a cluster to each observation and then proceeds to the update step, thus computing the initial mean to be the centroid of the cluster's randomly assigned points. The Forgy method tends to spread the initial means out, while Random Partition places all of them close to the center of the data set. According to Hamerly et al.,[11] the Random Partition method is generally preferable for algorithms such as the *k*-harmonic means and fuzzy *k*-means. For expectation maximization and standard *k*-means algorithms, the Forgy method of initialization is preferable. A comprehensive study by Celebi et al.,[12] however, found that popular initialization methods such as Forgy, Random Partition, and Maximin often perform poorly, whereas Bradley and Fayyad's approach[13] performs "consistently" in "the best group" and *k*-means++ performs "generally well".

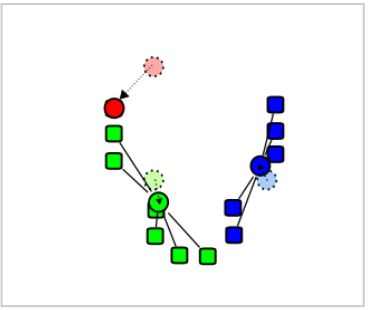
Demonstration of the standard algorithm



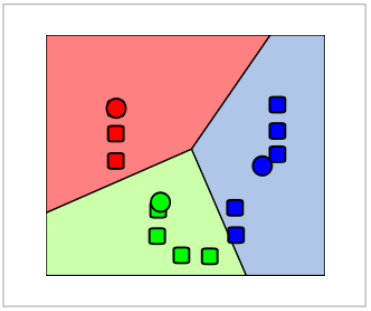
1. *k* initial "means" (in this case *k*=3) are randomly generated within the data domain (shown in color).



2. *k* clusters are created by associating every observation with the nearest mean. The partitions here represent the Voronoi diagram generated by the means.



3. The centroid of each of the *k* clusters becomes the new mean.



4. Steps 2 and 3 are repeated until convergence has been reached.

The algorithm does not guarantee convergence to the global optimum. The result may depend on the initial clusters. As the algorithm is usually fast, it is common to run it multiple times with different starting conditions. However, worst-case performance can be slow: in particular certain point sets, even in two dimensions, converge in exponential time, that is $2^{\Omega(n)}$.[14] These point sets do not seem to arise in practice: this is corroborated by the fact that the smoothed running time of *k*-means is polynomial.[15]

The "assignment" step is referred to as the "expectation step", while the "update step" is a maximization step, making this algorithm a variant of the *generalized* expectation–maximization algorithm.

Complexity

Finding the optimal solution to the *k*-means clustering problem for observations in *d* dimensions is:

- NP-hard in general Euclidean space (of *d* dimensions) even for two clusters,[16][17]
- NP-hard for a general number of clusters *k* even in the plane,[18]
- if *k* and *d* (the dimension) are fixed, the problem can be exactly solved in time $O(n^{dk+1})$, where *n* is the number of entities to be clustered.[19]

Thus, a variety of heuristic algorithms such as Lloyd's algorithm given above are generally used.

The running time of Lloyd's algorithm (and most variants) is $O(nkdi)$,[10][20] where:

- *n* is the number of *d*-dimensional vectors (to be clustered)

- *k* the number of clusters
- *i* the number of iterations needed until convergence.

On data that does have a clustering structure, the number of iterations until convergence is often small, and results only improve slightly after the first dozen iterations. Lloyd's algorithm is therefore often considered to be of "linear" complexity in practice, although it is in the worst case superpolynomial when performed until convergence.[21]

- In the worst-case, Lloyd's algorithm needs $i = 2^{\Omega(\sqrt{n})}$ iterations, so that the worst-case complexity of Lloyd's algorithm is superpolynomial.[21]
- Lloyd's *k*-means algorithm has polynomial smoothed running time. It is shown that[15] for arbitrary set of *n* points in $[0,1]^d$, if each point is independently perturbed by a normal distribution with mean 0 and variance σ^2, then the expected running time of *k*-means algorithm is bounded by $O(n^{34}k^{34}d^8\log^4(n)/\sigma^6)$, which is a polynomial in n, k, d and $1/\sigma$.
- Better bounds are proven for simple cases. For example, it is shown that the running time of *k*-means algorithm is bounded by $O(dn^4M^2)$ for *n* points in an integer lattice $\{1,\dots,M\}^d$.[22]

Lloyd's algorithm is the standard approach for this problem. However, it spends a lot of processing time computing the distances between each of the *k* cluster centers and the *n* data points. Since points usually stay in the same clusters after a few iterations, much of this work is unnecessary, making the naïve implementation very inefficient. Some implementations use caching and the triangle inequality in order to create bounds and accelerate Lloyd's algorithm.[23][10][24][25][26][27]

Optimal number of clusters

Finding the optimal number of clusters (*k*) for *k*-means clustering is a crucial step to ensure that the clustering results are meaningful and useful.[28] Several techniques are available to determine a suitable number of clusters. Here are some of commonly used methods:

- Elbow method (clustering): This method involves plotting the explained variation as a function of the number of clusters, and picking the elbow of the curve as the number of clusters to use.[29] However, the notion of an "elbow" is not well-defined and this is known to be unreliable.[30]
- Silhouette (clustering): Silhouette analysis measures the quality of clustering and provides an insight into the separation distance between the resulting clusters.[31] A higher silhouette score indicates that the object is well matched to its own cluster and poorly matched to neighboring clusters.
- Gap statistic: The Gap Statistic compares the total within intra-cluster variation for different values of k with their expected values under null reference distribution of the data.[32] The optimal k is the value that yields the largest gap statistic.
- Davies–Bouldin index: The Davies-Bouldin index is a measure of the how much separation there is between clusters.[33] Lower values of the Davies-Bouldin index indicate a model with better separation.
- Calinski-Harabasz index: This Index evaluates clusters based on their compactness and separation. The index is calculated using the ratio of between-cluster variance to within-cluster variance, with higher values indicate better-defined clusters.[34]
- Rand index: It calculates the proportion of agreement between the two clusters, considering both the pairs of elements that are correctly assigned to the same or different clusters.[35]

Higher values indicate greater similarity and better clustering quality. To provide a more accurate measure, the Adjusted Rand Index (ARI), introduced by Hubert and Arabie in 1985, corrects the Rand Index by adjusting for the expected similarity of all pairings due to chance.[36]

Variations

- Jenks natural breaks optimization: *k*-means applied to univariate data
- *k*-medians clustering uses the median in each dimension instead of the mean, and this way minimizes L_1 norm (Taxicab geometry).
- *k*-medoids (also: Partitioning Around Medoids, PAM) uses the medoid instead of the mean, and this way minimizes the sum of distances for *arbitrary* distance functions.
- Fuzzy C-Means Clustering is a soft version of *k*-means, where each data point has a fuzzy degree of belonging to each cluster.
- Gaussian mixture models trained with expectation–maximization algorithm (EM algorithm) maintains probabilistic assignments to clusters, instead of deterministic assignments, and multivariate Gaussian distributions instead of means.
- *k*-means++ chooses initial centers in a way that gives a provable upper bound on the WCSS objective.
- The filtering algorithm uses *k*-d trees to speed up each *k*-means step.[37]
- Some methods attempt to speed up each *k*-means step using the triangle inequality.[24][25][26][38][27]
- Escape local optima by swapping points between clusters.[10]
- The Spherical *k*-means clustering algorithm is suitable for textual data.[39]
- Hierarchical variants such as Bisecting *k*-means,[40] X-means clustering[41] and G-means clustering[42] repeatedly split clusters to build a hierarchy, and can also try to automatically determine the optimal number of clusters in a dataset.
- Internal cluster evaluation measures such as cluster silhouette can be helpful at determining the number of clusters.
- Minkowski weighted *k*-means automatically calculates cluster specific feature weights, supporting the intuitive idea that a feature may have different degrees of relevance at different features.[43] These weights can also be used to re-scale a given data set, increasing the likelihood of a cluster validity index to be optimized at the expected number of clusters.[44]
- Mini-batch *k*-means: *k*-means variation using "mini batch" samples for data sets that do not fit into memory.[45]
- Otsu's method

Hartigan–Wong method

Hartigan and Wong's method[10] provides a variation of *k*-means algorithm which progresses towards a local minimum of the minimum sum-of-squares problem with different solution updates. The method is a local search that iteratively attempts to relocate a sample into a different cluster as long as this process improves the objective function. When no sample can be relocated into a different cluster with an improvement of the objective, the method stops (in a local minimum). In a similar way as the classical *k*-means, the approach remains a heuristic since it does not necessarily guarantee that the final solution is globally optimum.

Let $\varphi(S_j)$ be the individual cost of S_j defined by $\sum_{x \in S_j} (x - \mu_j)^2$, with μ_j the center of the cluster.

Assignment step
: Hartigan and Wong's method starts by partitioning the points into random clusters $\{S_j\}_{j \in \{1,\cdots k\}}$.

Update step
: Next it determines the $n, m \in \{1, \ldots, k\}$ and $x \in S_n$ for which the following function reaches a maximum

$$\Delta(m, n, x) = \varphi(S_n) + \varphi(S_m) - \varphi(S_n \setminus \{x\}) - \varphi(S_m \cup \{x\}).$$

: For the x, n, m that reach this maximum, x moves from the cluster S_n to the cluster S_m.

Termination
: The algorithm terminates once $\Delta(m, n, x)$ is less than zero for all x, n, m.

Different move acceptance strategies can be used. In a *first-improvement* strategy, any improving relocation can be applied, whereas in a *best-improvement* strategy, all possible relocations are iteratively tested and only the best is applied at each iteration. The former approach favors speed, whether the latter approach generally favors solution quality at the expense of additional computational time. The function Δ used to calculate the result of a relocation can also be efficiently evaluated by using equality[46]

$$\Delta(x, n, m) = \frac{|S_n|}{|S_n \mid -1} \cdot \|\mu_n - x\|^2 - \frac{|S_m|}{|S_m \mid +1} \cdot \|\mu_m - x\|^2.$$

Global optimization and meta-heuristics

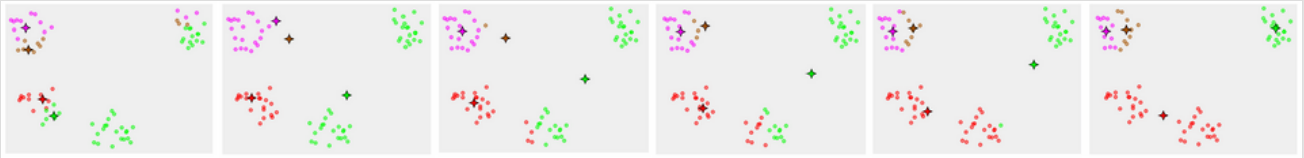
The classical *k*-means algorithm and its variations are known to only converge to local minima of the minimum-sum-of-squares clustering problem defined as

$$\operatorname*{arg\,min}_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2.$$

Many studies have attempted to improve the convergence behavior of the algorithm and maximize the chances of attaining the global optimum (or at least, local minima of better quality). Initialization and restart techniques discussed in the previous sections are one alternative to find better solutions. More recently, global optimization algorithms based on branch-and-bound and semidefinite programming have produced *provenly optimal* solutions for datasets with up to 4,177 entities and 20,531 features.[47] As expected, due to the NP-hardness of the subjacent optimization problem, the computational time of optimal algorithms for *k*-means quickly increases beyond this size. Optimal solutions for small- and medium-scale still remain valuable as a benchmark tool, to evaluate the quality of other heuristics. To find high-quality local minima within a controlled computational time but without optimality guarantees, other works have explored metaheuristics and other global optimization techniques, e.g., based on incremental approaches and convex optimization,[48] random swaps[49] (i.e., iterated local search),

variable neighborhood search[50] and genetic algorithms.[51][52] It is indeed known that finding better local minima of the minimum sum-of-squares clustering problem can make the difference between failure and success to recover cluster structures in feature spaces of high dimension.[52]

Discussion

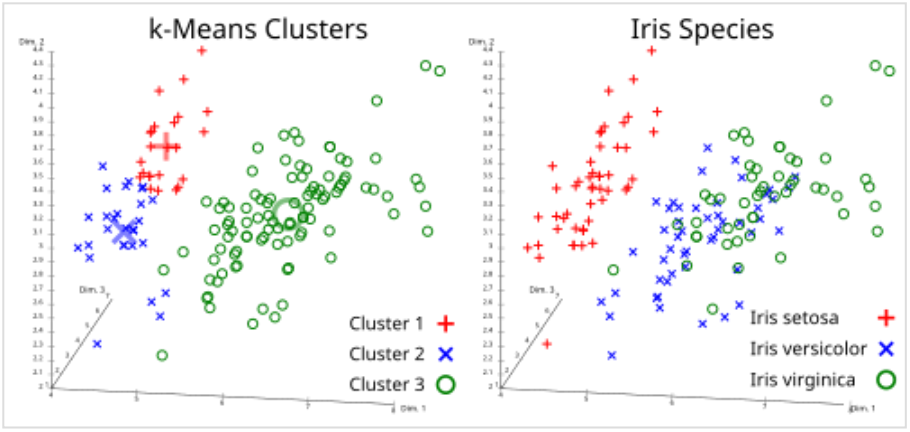


A typical example of the *k*-means convergence to a local minimum. In this example, the result of *k*-means clustering (the right figure) contradicts the obvious cluster structure of the data set. The small circles are the data points, the four ray stars are the centroids (means). The initial configuration is on the left figure. The algorithm converges after five iterations presented on the figures, from the left to the right. The illustration was prepared with the Mirkes Java applet.[53]

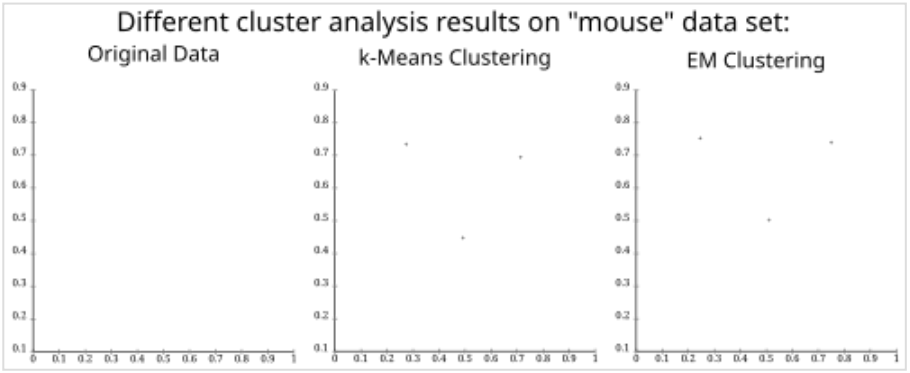
Three key features of *k*-means that make it efficient are often regarded as its biggest drawbacks:

- Euclidean distance is used as a metric and variance is used as a measure of cluster scatter.
- The number of clusters *k* is an input parameter: an inappropriate choice of *k* may yield poor results. That is why, when performing *k*-means, it is important to run diagnostic checks for determining the number of clusters in the data set.
- Convergence to a local minimum may produce counterintuitive ("wrong") results (see example in Fig.).

A key limitation of *k*-means is its cluster model. The concept is based on spherical clusters that are separable so that the mean converges towards the cluster center. The clusters are



k-means clustering result for the Iris flower data set and actual species visualized using ELKI. Cluster means are marked using larger, semi-transparent symbols.



k-means clustering vs. EM clustering on an artificial dataset ("mouse"). The tendency of *k*-means to produce equal-sized clusters leads to bad results here, while EM benefits from the Gaussian distributions with different radius present in the data set.

expected to be of similar size, so that the assignment to the nearest cluster center is the correct assignment. When for example applying *k*-means with a value of $k = 3$ onto the well-known Iris flower data set, the result often fails to separate the three Iris species contained in the data set. With $k = 2$, the two visible clusters (one containing two species) will be discovered, whereas with $k = 3$ one of the two clusters will be split into two even parts. In fact, $k = 2$ is more appropriate for this data set, despite the data set's containing 3 *classes*. As with any other clustering algorithm, the *k*-means result makes assumptions that the data satisfy certain criteria. It works well on some data sets, and fails on others.

The result of *k*-means can be seen as the Voronoi cells of the cluster means. Since data is split halfway between cluster means, this can lead to suboptimal splits as can be seen in the "mouse" example. The Gaussian models used by the expectation–maximization algorithm (arguably a generalization of *k*-means) are more flexible by having both variances and covariances. The EM result is thus able to accommodate clusters of variable size much better than *k*-means as well as correlated clusters (not in this example). In counterpart, EM requires the optimization of a larger number of free parameters and poses some methodological issues due to vanishing clusters or badly-conditioned covariance matrices. *k*-means is closely related to nonparametric Bayesian modeling.[54]

Applications

k-means clustering is rather easy to apply to even large data sets, particularly when using heuristics such as Lloyd's algorithm. It has been successfully used in market segmentation, computer vision, and astronomy among many other domains. It often is used as a preprocessing step for other algorithms, for example to find a starting configuration.

Biology

In biological applications, the limitations of standard *k*-means clustering are often addressed by adapting the distance measures used and objective functions to better reflect the structure of experimental data. Rather than relying on Euclidean distance and variance-based measures of cluster scatter, alternative similarity metrics such as the Jaccard index are sometimes used when analyzing genomic datasets with k-means. In these contexts, clustering may be performed directly on distance matrices, and medoids (representative data points that have the smallest average distance to every other point within a cluster) may be selected as cluster centers instead of arithmetic means. This allows clustering methods to capture patterns of shared genomic features or co-occurrence that are not well represented in a traditional Euclidean analysis.[55]

Clustering is widely used in biological data analysis to identify patterns in high-dimensional datasets such as gene expression profiles and genomic interaction matrices. Techniques such as *k*-means clustering partition observations into groups based on similarity, allowing researchers to detect structure in complex datasets. In gene expression studies, clustering is commonly used to group genes with similar expression profiles, often revealing co-regulated genes and underlying biological interactions.[56]

In addition to gene expression analysis, clustering approaches are applied to chromatin interaction data to identify regions of coordinated activity. Similarity measures such as the Jaccard index can be used to quantify shared features between observations, and clustering of the resulting distance matrices can reveal structural organization within the genome. These patterns are commonly visualized using heatmaps,

where clustered regions correspond to domains of interaction or co-segregation, providing insight into the regulatory organization of the genome.[55] In such analyses, clustering of similarity or co-detection matrices can reveal groups of genomic windows that exhibit coordinated behavior across experimental conditions.[55]

Clustering methods provide a practical method for uncovering biologically meaningful structure from complex datasets. By grouping observations with similar patterns of genomic detection, expression, or interaction, these techniques can reveal functional relationships, spatial organization within the nucleus, and regulatory behavior. In many applications, the resulting clusters correspond to distinct biological states or structural domains, further illustrating the broad applicability of k-means clustering.[56]



Heatmap of clustered genomic co-detection data. Each row represents a genomic window and each column represents a nuclear profile, with white indicating detected regions and black indicating absence. The clustered arrangement reveals groups of regions with similar co-detection patterns, illustrating how clustering can identify coordinated genomic behavior.

Vector quantization

Vector quantization, a technique commonly used in signal processing and computer graphics, involves reducing the color palette of an image to a fixed number of colors, known as *k*. One popular method for achieving vector quantization is through *k*-means clustering. In this process, *k*-means is applied to the color space of an image to partition it into k clusters, with each cluster representing a distinct color in the image. This technique is particularly useful in image segmentation tasks, where it helps identify and group similar colors together.

Example: In the field of computer graphics, *k*-means clustering is often employed for color quantization in image compression. By reducing the number of colors used to represent an image, file sizes can be significantly reduced without significant loss of visual quality. For instance, consider an image with millions of colors. By applying *k*-means clustering with *k* set to a smaller number, the image can be represented using a more limited color palette, resulting in a compressed version that consumes less storage space and bandwidth. Other uses of vector quantization include non-random sampling, as *k*-means can easily be used to choose *k* different but prototypical objects from a large data set for further analysis.

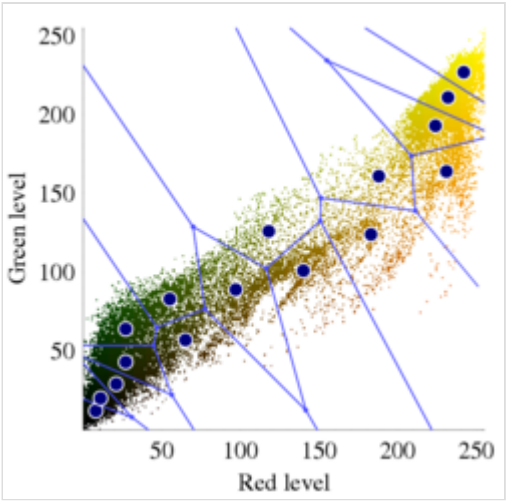

Example image with only red and green channel (for illustration purposes)

Cluster analysis

Cluster analysis, a fundamental task in data mining and machine learning, involves grouping a set of data points into clusters based on their similarity. *k*-means clustering is a popular algorithm used for partitioning data into k clusters, where each cluster is represented by its centroid.

However, the pure *k*-means algorithm is not very flexible, and as such is of limited use (except for when vector quantization as above is actually the desired use case). In particular, the parameter *k* is known to be hard to choose (as discussed above) when not given by external constraints. Another limitation is that it cannot be used with arbitrary distance functions or on non-numerical data. For these use cases, many other algorithms are superior.

Example: In marketing, *k*-means clustering is frequently employed for market segmentation, where customers with similar characteristics or behaviors are grouped together. For instance, a retail company may use *k*-means clustering to segment its customer base into distinct groups based on factors such as purchasing behavior, demographics, and geographic location. These customer segments can then be targeted with tailored marketing strategies and product offerings to maximize sales and customer satisfaction.



Vector quantization of colors present in the image above into Voronoi cells using *k*-means

Feature learning

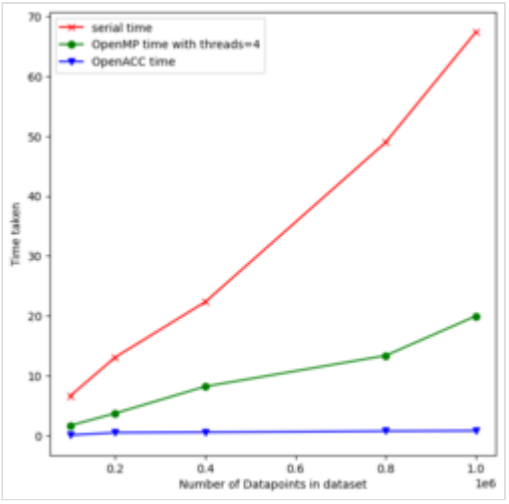
k-means clustering has been used as a feature learning (or dictionary learning) step, in either (semi-)supervised learning or unsupervised learning.[57] The basic approach is first to train a *k*-means clustering representation, using the input training data (which need not be labelled). Then, to project any input datum into the new feature space, an "encoding" function, such as the thresholded matrix-product of the datum with the centroid locations, computes the distance from the datum to each centroid, or simply an indicator function for the nearest centroid,[57][58] or some smooth transformation of the distance.[59] Alternatively, transforming the sample-cluster distance through a Gaussian RBF, obtains the hidden layer of a radial basis function network.[60]

This use of *k*-means has been successfully combined with simple, linear classifiers for semi-supervised learning in NLP (specifically for named-entity recognition)[61] and in computer vision. On an object recognition task, it was found to exhibit comparable performance with more sophisticated feature learning approaches such as autoencoders and restricted Boltzmann machines.[59] However, it generally requires more data, for equivalent performance, because each data point only contributes to one "feature".[57]

Example: In natural language processing (NLP), *k*-means clustering has been integrated with simple linear classifiers for semi-supervised learning tasks such as named-entity recognition (NER). By first clustering unlabeled text data using *k*-means, meaningful features can be extracted to improve the performance of NER models. For instance, *k*-means clustering can be applied to identify clusters of words or phrases that frequently co-occur in the input text, which can then be used as features for training the NER model. This approach has been shown to achieve comparable performance with more complex feature learning techniques such as autoencoders and restricted Boltzmann machines, albeit with a greater requirement for labeled data.

Big Data Parallelization

Standard Lloyd's algorithm is inherently sequential. It has an effective time complexity of O(nkdi), which becomes a bottleneck as the number of data points (n) or dimensions (d) increases. This can be inefficient when used with massive, multidimensional data sets like social networks. Various parallelization strategies have been developed to distribute the workload across multi-core processors, GPUs, and distributed clusters.



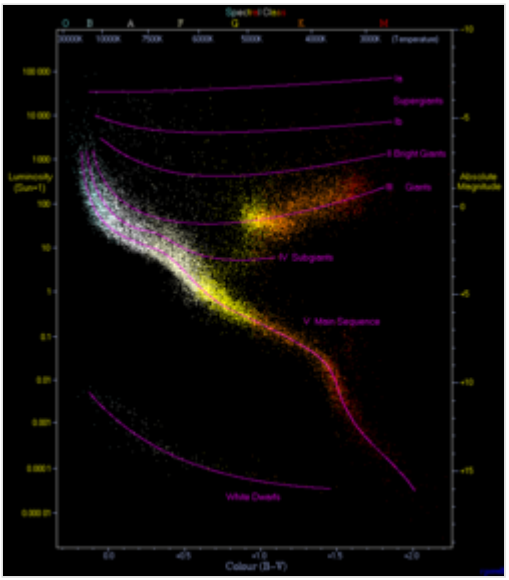
Execution time of the k-means clustering algorithm as the dataset size grows using OpenMP, OpenACC, and serial approaches.

- **Shared Memory Model (OpenMP):** This approach divides the work between CPU threads for the reassignment step of the process. These threads independently calculate distances to centroids. Partial sums and counts from each thread are aggregated to compute the new centroids for the next iteration.[62]
- **GPU Parallelization (OpenACC):** Unlike the OpenMP implementation, the OpenACC program's parallel directive is called at each necessary step instead of at the very beginning. This exploits the SIMT (Single Instruction, Multiple Threads) architecture of GPUs to perform thousands of distance calculations simultaneously.[62]

We see that in both parallel versions; efficiency is greatly increased for large datasets. This speedup is significant in the OpenACC implementation, due to the massively parallel nature of GPU architectures.

Astronomy

Modern astronomical surveys like APOGEE and the Gaia catalog collect spectroscopic and photometric data for millions of stars, making manual classification impractical at scale. *K*-means clustering has been applied to this data to automatically identify distinct stellar populations based on properties like luminosity, temperature, color, and chemical abundance.[63]


A Hertzsprung-Russell diagram plotting stars by luminosity and spectral type, the kind of feature data used in k-means stellar classification.

One major application is identifying stellar populations through chemical tagging. Stars that form together in the same cluster share similar chemical compositions, so clustering algorithms applied to abundance data can recover these groupings even after the stars have spread throughout the galaxy. Studies using *k*-means on APOGEE infrared spectra measured abundances of up to 13 elements and successfully distinguished known star clusters from surrounding field stars.[63] This has also revealed populations with unusual abundance patterns that would be difficult to catch through manual inspection.

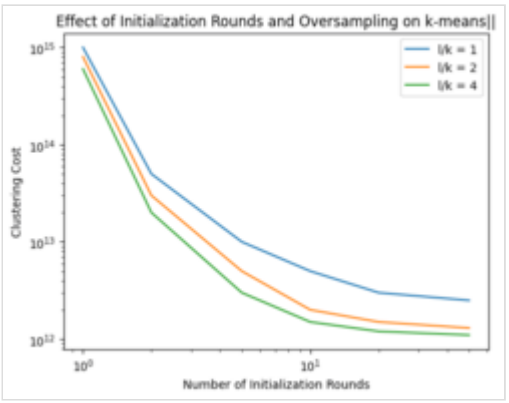
K-means has also been combined with dimensionality reduction techniques to classify large numbers of unlabeled survey objects by separating stars, galaxies, and quasars based on their observed light properties. Labels from confirmed examples are then propagated to nearby cluster members. This approach has achieved accuracy comparable to fully supervised methods while requiring far fewer labeled examples, which makes it well suited for the scale of modern sky surveys.[64]

K-means++ and Scalable Initialization

Proper initialization is very important when it comes to finding global optimum in a clustering algorithm. Random initialization is often used for k means clustering, but this can result in poor cluster quality.[65] This is why the k-means++ initialization algorithm was developed.[65] The algorithm attempts to find initial centers that are as close to possible to the global optimum, reducing computing time and producing more accurate clusters.[65]

The k-means++ algorithm works by spreading out the initial cluster centers. It selects the first center uniformly at random from the dataset. Each subsequent center is chosen stochastically, with the probability of a point being selected proportional to its contribution to the overall error.[65] This method achieves an O(log k) approximation of the optimum solution.[65]

The primary drawback of standard k-means++ is its inherent sequential nature. To find k initial centers, the algorithm must make k separate passes over the data, as the selection of each new center depends on the previous choices.[66] This becomes prohibitively slow when clustering massive datasets into a large number of clusters.[66]



Effect of the number of initialization rounds and oversampling factor (ℓ) on clustering cost in k-means||. The plot shows that clustering quality improves rapidly during the first few rounds, with diminishing returns as the number of rounds increases.

To overcome these limitations, researchers developed k-means||, a parallel version of k-means++.[66] Instead of sampling a single point per pass, k-means|| uses an oversampling factor $\ell = \Omega(k)$ to sample multiple points in each round.[66] The algorithm drastically reduces the number of passes required, obtaining a nearly optimal set of centers with a time complexity of O(log n).[66] In practice, as few as five rounds are needed to reach a high-quality solution. After several rounds, the algorithm typically has O(log n) intermediate centers.[66] These are then assigned weights based on how many points are close to them and re-clustered, often using standard k-means++, to reduce the final set to exact k centers.[66]

Recent developments

Recent advancements in the application of *k*-means clustering have explored the integration of *k*-means clustering with deep learning methods, such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), to enhance the performance of various tasks in computer vision, natural language processing, and other domains.

Relation to other algorithms

Gaussian mixture model

The slow "standard algorithm" for *k*-means clustering, and its associated expectation–maximization algorithm, is a special case of a Gaussian mixture model, specifically, the limiting case when fixing all covariances to be diagonal, equal and have infinitesimal small variance.[67]:850 Instead of small variances, a hard cluster assignment can also be used to show another equivalence of *k*-means clustering to a special case of "hard" Gaussian mixture modelling.[68]:354,11.4.2.5 This does not mean that it is efficient to use Gaussian mixture modelling to compute *k*-means, but just that there is a theoretical relationship, and that Gaussian mixture modelling can be interpreted as a generalization of *k*-means; on the contrary, it has been suggested to use *k*-means clustering to find starting points for Gaussian mixture modelling on difficult data.[67]:849

k-SVD

Another generalization of the *k*-means algorithm is the *k*-SVD algorithm, which estimates data points as a sparse linear combination of "codebook vectors". *k*-means corresponds to the special case of using a single codebook vector, with a weight of 1.[69]

Principal component analysis

The relaxed solution of *k*-means clustering, specified by the cluster indicators, is given by principal component analysis (PCA).[70][71] The intuition is that *k*-means describe spherically shaped (ball-like) clusters. If the data has 2 clusters, the line connecting the two centroids is the best 1-dimensional projection direction, which is also the first PCA direction. Cutting the line at the center of mass separates the clusters (this is the continuous relaxation of the discrete cluster indicator). If the data have three clusters, the 2-dimensional plane spanned by three cluster centroids is the best 2-D projection. This plane is also defined by the first two PCA dimensions. Well-separated clusters are effectively modelled by ball-shaped clusters and thus discovered by *k*-means. Non-ball-shaped clusters are hard to separate when they are close. For example, two half-moon shaped clusters intertwined in space do not separate well when projected onto PCA subspace. *k*-means should not be expected to do well on this data.[72] It is straightforward to produce counterexamples to the statement that the cluster centroid subspace is spanned by the principal directions.[73]

Mean shift clustering

Basic mean shift clustering algorithms maintain a set of data points the same size as the input data set. Initially, this set is copied from the input set. All points are then iteratively moved towards the mean of the points surrounding them. By contrast, *k*-means restricts the set of clusters to *k* clusters, usually much less than the number of points in the input data set, using the mean of all points in the prior cluster that are closer to that point than any other for the centroid (e.g. within the Voronoi partition of each updating point). A mean shift algorithm that is similar then to *k*-means, called *likelihood mean shift*, replaces the set of points undergoing replacement by the mean of all points in the input set that are within a given

distance of the changing set.[74] An advantage of mean shift clustering over *k*-means is the detection of an arbitrary number of clusters in the data set, as there is not a parameter determining the number of clusters. Mean shift can be much slower than *k*-means, and still requires selection of a bandwidth parameter.

Independent component analysis

Under sparsity assumptions and when input data is pre-processed with the whitening transformation, *k*-means produces the solution to the linear independent component analysis (ICA) task. This aids in explaining the successful application of *k*-means to feature learning.[75]

Bilateral filtering

k-means implicitly assumes that the ordering of the input data set does not matter. The bilateral filter is similar to *k*-means and mean shift in that it maintains a set of data points that are iteratively replaced by means. However, the bilateral filter restricts the calculation of the (kernel weighted) mean to include only points that are close in the ordering of the input data.[74] This makes it applicable to problems such as image denoising, where the spatial arrangement of pixels in an image is of critical importance.

Similar problems

The set of squared error minimizing cluster functions also includes the *k*-medoids algorithm, an approach which forces the center point of each cluster to be one of the actual points, i.e., it uses medoids in place of centroids.

Software implementations

Different implementations of the algorithm exhibit performance differences, with the fastest on a test data set finishing in 10 seconds, the slowest taking 25,988 seconds (~7 hours).[1] The differences can be attributed to implementation quality, language and compiler differences, different termination criteria and precision levels, and the use of indexes for acceleration.

Free Software/Open Source

The following implementations are available under Free/Open Source Software licenses, with publicly available source code.

- Accord.NET contains C# implementations for *k*-means, *k*-means++ and *k*-modes.
- ALGLIB contains parallelized C++ and C# implementations for *k*-means and *k*-means++.
- AOSP contains a Java implementation for *k*-means.
- CrimeStat implements two spatial *k*-means algorithms, one of which allows the user to define the starting locations.
- ELKI contains *k*-means (with Lloyd and MacQueen iteration, along with different initializations such as *k*-means++ initialization) and various more advanced clustering algorithms.
- Smile contains *k*-means and various more other algorithms and results visualization (for java, kotlin and scala).

- Julia contains a *k*-means implementation in the JuliaStats Clustering package.
- KNIME contains nodes for *k*-means and *k*-medoids.
- Mahout contains a MapReduce based *k*-means.
- mlpack contains a C++ implementation of *k*-means.
- Octave contains *k*-means.
- OpenCV contains a *k*-means implementation.
- Orange includes a component for *k*-means clustering with automatic selection of *k* and cluster silhouette scoring.
- PSPP contains *k*-means, The QUICK CLUSTER command performs *k*-means clustering on the dataset.
- R contains three *k*-means variations.
- SciPy and scikit-learn contain multiple *k*-means implementations.
- Spark MLlib implements a distributed *k*-means algorithm.
- Torch contains an *unsup* package that provides *k*-means clustering.
- Weka contains *k*-means and *x*-means.

Proprietary

The following implementations are available under proprietary license terms, and may not have publicly available source code.

- Ayasdi
- Mathematica
- MATLAB
- OriginPro
- RapidMiner
- SAP HANA
- SAS
- SPSS
- Stata

See also

- K-medoids
- BFR algorithm
- Centroidal Voronoi tessellation
- Cluster analysis
- DBSCAN
- Head/tail breaks
- *k q*-flats
- *k*-means++
- Linde–Buzo–Gray algorithm
- Self-organizing map

References

1. Kriegel, Hans-Peter; Schubert, Erich; Zimek, Arthur (2016). "The (black) art of runtime evaluation: Are we comparing algorithms or implementations?". *Knowledge and Information Systems*. **52** (2): 341–378. doi:10.1007/s10115-016-1004-2 (https://doi.org/10.1007%2Fs10115-016-1004-2). ISSN 0219-1377 (https://search.worldcat.org/issn/0219-1377). S2CID 40772241 (https://api.semanticscholar.org/CorpusID:40772241).

2. MacQueen, J. B. (1967). *Some Methods for classification and Analysis of Multivariate Observations* (http://projecteuclid.org/euclid.bsmsp/1200512992). Proceedings of 5th Berkeley Symposium on Mathematical Statistics and Probability. Vol. 1. University of California Press. pp. 281–297. MR 0214227 (https://mathscinet.ams.org/mathscinet-getitem?mr=0214227). Zbl 0214.46201 (https://zbmath.org/?format=complete&q=an:0214.46201). Retrieved 2009-04-07.
3. Steinhaus, Hugo (1957). "Sur la division des corps matériels en parties". *Bull. Acad. Polon. Sci.* (in French). **4** (12): 801–804. MR 0090073 (https://mathscinet.ams.org/mathscinet-getitem?mr=0090073). Zbl 0079.16403 (https://zbmath.org/?format=complete&q=an:0079.16403).
4. Lloyd, Stuart P. (1957). "Least square quantization in PCM". *Bell Telephone Laboratories Paper*. Published in journal much later: Lloyd, Stuart P. (1982). "Least squares quantization in PCM" (http://www.cs.toronto.edu/~roweis/csc2515-2006/readings/lloyd57.pdf) (PDF). *IEEE Transactions on Information Theory*. **28** (2): 129–137. Bibcode:1982ITIT...28..129L (https://ui.adsabs.harvard.edu/abs/1982ITIT...28..129L). doi:10.1109/TIT.1982.1056489 (https://doi.org/10.1109%2FTIT.1982.1056489). S2CID 10833328 (https://api.semanticscholar.org/CorpusID:10833328). Retrieved 2009-04-15.
5. Forgy, Edward W. (1965). "Cluster analysis of multivariate data: efficiency versus interpretability of classifications". *Biometrics*. **21** (3): 768–769. JSTOR 2528559 (https://www.jstor.org/stable/2528559).
6. Jain, A. K. (2010). "Data clustering: 50 years beyond k-means". *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **31** (11): 651–666. Bibcode:2010PaReL..31..651J (https://ui.adsabs.harvard.edu/abs/2010PaReL..31..651J). doi:10.1016/j.patrec.2009.09.011 (https://doi.org/10.1016%2Fj.patrec.2009.09.011).
7. Pelleg, Dan; Moore, Andrew (1999). "Accelerating exact *k* -means algorithms with geometric reasoning" (http://portal.acm.org/citation.cfm?doid=312129.312248). *Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining*. San Diego, California, United States: ACM Press. pp. 277–281. doi:10.1145/312129.312248 (https://doi.org/10.1145%2F312129.312248). ISBN 978-1-58113-143-7. S2CID 13907420 (https://api.semanticscholar.org/CorpusID:13907420).
8. MacKay, David (2003). "Chapter 20. An Example Inference Task: Clustering" (http://www.inference.phy.cam.ac.uk/mackay/itprnn/ps/284.292.pdf) (PDF). *Information Theory, Inference and Learning Algorithms* (http://www.inference.phy.cam.ac.uk/mackay/itila/book.html). Cambridge University Press. pp. 284–292. ISBN 978-0-521-64298-9. MR 2012999 (https://mathscinet.ams.org/mathscinet-getitem?mr=2012999).
9. Since the square root is a monotone function, this also is the minimum Euclidean distance assignment.
10. Hartigan, J. A.; Wong, M. A. (1979). "Algorithm AS 136: A *k*-Means Clustering Algorithm". *Journal of the Royal Statistical Society, Series C*. **28** (1): 100–108. JSTOR 2346830 (https://www.jstor.org/stable/2346830).
11. Hamerly, Greg; Elkan, Charles (2002). "Alternatives to the *k*-means algorithm that find better clusterings" (http://people.csail.mit.edu/tieu/notebook/kmeans/15_p600-hamerly.pdf) (PDF). *Proceedings of the eleventh international conference on Information and knowledge management (CIKM)*.
12. Celebi, M. E.; Kingravi, H. A.; Vela, P. A. (2013). "A comparative study of efficient initialization methods for the *k*-means clustering algorithm". *Expert Systems with Applications*. **40** (1): 200–210. arXiv:1209.1960 (https://arxiv.org/abs/1209.1960). doi:10.1016/j.eswa.2012.07.021 (https://doi.org/10.1016%2Fj.eswa.2012.07.021). S2CID 6954668 (https://api.semanticscholar.org/CorpusID:6954668).
13. Bradley, Paul S.; Fayyad, Usama M. (1998). "Refining Initial Points for *k*-Means Clustering". *Proceedings of the Fifteenth International Conference on Machine Learning*.

14. Vattani, A. (2011). "k-means requires exponentially many iterations even in the plane" (http://cseweb.ucsd.edu/users/avattani/papers/kmeans-journal.pdf) (PDF). *Discrete and Computational Geometry*. **45** (4): 596–616. doi:10.1007/s00454-011-9340-1 (https://doi.org/10.1007%2Fs00454-011-9340-1). S2CID 42683406 (https://api.semanticscholar.org/CorpusID:42683406).
15. Arthur, David; Manthey, B.; Roeglin, H. (2009). "k-means has polynomial smoothed complexity". *Proceedings of the 50th Symposium on Foundations of Computer Science (FOCS)*. arXiv:0904.1113 (https://arxiv.org/abs/0904.1113).
16. Aloise, D.; Deshpande, A.; Hansen, P.; Popat, P. (2009). "NP-hardness of Euclidean sum-of-squares clustering" (https://doi.org/10.1007%2Fs10994-009-5103-0). *Machine Learning*. **75** (2): 245–249. Bibcode:2009MLear..75..245A (https://ui.adsabs.harvard.edu/abs/2009MLear..75..245A). doi:10.1007/s10994-009-5103-0 (https://doi.org/10.1007%2Fs10994-009-5103-0).
17. Dasgupta, S.; Freund, Y. (July 2009). "Random Projection Trees for Vector Quantization". *IEEE Transactions on Information Theory*. **55** (7): 3229–42. arXiv:0805.1390 (https://arxiv.org/abs/0805.1390). Bibcode:2009ITIT...55.3229D (https://ui.adsabs.harvard.edu/abs/2009ITIT...55.3229D). doi:10.1109/TIT.2009.2021326 (https://doi.org/10.1109%2FTIT.2009.2021326). S2CID 666114 (https://api.semanticscholar.org/CorpusID:666114).
18. Mahajan, Meena; Nimbhorkar, Prajakta; Varadarajan, Kasturi (2009). "The Planar k-Means Problem is NP-Hard". *WALCOM: Algorithms and Computation*. Lecture Notes in Computer Science. Vol. 5431. pp. 274–285. doi:10.1007/978-3-642-00202-1_24 (https://doi.org/10.1007%2F978-3-642-00202-1_24). ISBN 978-3-642-00201-4.
19. Inaba, M.; Katoh, N.; Imai, H. (1994). *Applications of weighted Voronoi diagrams and randomization to variance-based* k-*clustering*. Proceedings of 10th ACM Symposium on Computational Geometry. pp. 332–9. doi:10.1145/177424.178042 (https://doi.org/10.1145%2F177424.178042).
20. Manning, Christopher D.; Raghavan, Prabhakar; Schütze, Hinrich (2008). *Introduction to information retrieval*. Cambridge University Press. ISBN 978-0-521-86571-5. OCLC 190786122 (https://search.worldcat.org/oclc/190786122).
21. Arthur, David; Vassilvitskii, Sergei (2006-01-01). "How slow is the *k* -means method?". *Proceedings of the twenty-second annual symposium on Computational geometry*. SCG '06. ACM. pp. 144–153. doi:10.1145/1137856.1137880 (https://doi.org/10.1145%2F1137856.1137880). ISBN 978-1-59593-340-9. S2CID 3084311 (https://api.semanticscholar.org/CorpusID:3084311).
22. Bhowmick, Abhishek (2009). "A theoretical analysis of Lloyd's algorithm for *k*-means clustering" (https://web.archive.org/web/20151208140946/https://gautam5.cse.iitk.ac.in/opencs/sites/default/files/final.pdf) (PDF). Archived from the original (https://gautam5.cse.iitk.ac.in/opencs/sites/default/files/final.pdf) (PDF) on 2015-12-08. See also here (https://www.researchgate.net/publication/267854906_A_theoretical_analysis_of_Lloyd's_algorithm_for_k-means_clustering).
23. Ding, Yufei; Zhao, Yue; Shen, Xipeng; Musuvathi, Madan; Mytkowiczyear, Todd. "Yinyang K-Means: A Drop-In Replacement of the Classic K-Means with Consistent Speedup" (https://research.csc.ncsu.edu/picture/publications/papers/icml15.pdf) (PDF). *Proceedings of the Thirty Second International Conference on Machine Learning (ICML)*.
24. Phillips, Steven J. (2002). "Acceleration of K-Means and Related Clustering Algorithms". In Mount, David M.; Stein, Clifford (eds.). *Acceleration of* k-*Means and Related Clustering Algorithms*. Lecture Notes in Computer Science. Vol. 2409. Springer. pp. 166–177. doi:10.1007/3-540-45643-0_13 (https://doi.org/10.1007%2F3-540-45643-0_13). ISBN 978-3-540-43977-6.
25. Elkan, Charles (2003). "Using the triangle inequality to accelerate *k*-means" (http://www-cse.ucsd.edu/~elkan/kmeansicml03.pdf) (PDF). *Proceedings of the Twentieth International Conference on Machine Learning (ICML)*.

26. Hamerly, Greg (2010). "Making k-means even faster". *Proceedings of the 2010 SIAM International Conference on Data Mining*. pp. 130–140. doi:10.1137/1.9781611972801.12 (https://doi.org/10.1137%2F1.9781611972801.12). ISBN 978-0-89871-703-7.
27. Hamerly, Greg; Drake, Jonathan (2015). "Accelerating Lloyd's Algorithm for k-Means Clustering". *Partitional Clustering Algorithms*. pp. 41–78. doi:10.1007/978-3-319-09259-1_2 (https://doi.org/10.1007%2F978-3-319-09259-1_2). ISBN 978-3-319-09258-4.
28. Ikotun, Abiodun M.; Ezugwu, Absalom E.; Abualigah, Laith; Abuhaija, Belal; Heming, Jia (2023). "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data". *Information Sciences*. **622**: 178–210. doi:10.1016/j.ins.2022.11.139 (https://doi.org/10.1016%2Fj.ins.2022.11.139).
29. Thorndike, Robert L. (1953). "Who Belongs in the Family?". *Psychometrika*. **18** (4): 267–276. doi:10.1007/BF02289263 (https://doi.org/10.1007%2FBF02289263).
30. Schubert, Erich (2023). "Stop using the elbow criterion for k-means and how to choose the number of clusters instead". *ACM SIGKDD Explorations Newsletter*. **25**: 36–42. arXiv:2212.12189 (https://arxiv.org/abs/2212.12189). doi:10.1145/3606274.3606278 (https://doi.org/10.1145%2F3606274.3606278).
31. Rousseeuw, Peter J. (1987). "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis". *Journal of Computational and Applied Mathematics*. **20**: 53–65. doi:10.1016/0377-0427(87)90125-7 (https://doi.org/10.1016%2F0377-0427%2887%2990125-7).
32. Tibshirani, Robert; Walther, Guenther; Hastie, Trevor (2001). "Estimating the Number of Clusters in a Data Set Via the Gap Statistic". *Journal of the Royal Statistical Society Series B: Statistical Methodology*. **63** (2): 411–423. doi:10.1111/1467-9868.00293 (https://doi.org/10.1111%2F1467-9868.00293).
33. Davies, David L.; Bouldin, Donald W. (1979). "A Cluster Separation Measure". *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **1** (2): 224–227. Bibcode:1979ITPAM...1..224D (https://ui.adsabs.harvard.edu/abs/1979ITPAM...1..224D). doi:10.1109/TPAMI.1979.4766909 (https://doi.org/10.1109%2FTPAMI.1979.4766909). PMID 21868852 (https://pubmed.ncbi.nlm.nih.gov/21868852).
34. Caliński, T.; Harabasz, J. (1974). "A dendrite method for cluster analysis". *Communications in Statistics*. **3**: 1–27. doi:10.1080/03610927408827101 (https://doi.org/10.1080%2F03610927408827101).
35. Rand, William M. (1971). "Objective Criteria for the Evaluation of Clustering Methods". *Journal of the American Statistical Association*. **66** (336): 846–850. doi:10.2307/2284239 (https://doi.org/10.2307%2F2284239). JSTOR 2284239 (https://www.jstor.org/stable/2284239).
36. Hubert, Lawrence; Arabie, Phipps (1985). "Comparing partitions". *Journal of Classification*. **2**: 193–218. doi:10.1007/BF01908075 (https://doi.org/10.1007%2FBF01908075).
37. Kanungo, Tapas; Mount, David M.; Netanyahu, Nathan S.; Piatko, Christine D.; Silverman, Ruth; Wu, Angela Y. (2002). "An efficient *k*-means clustering algorithm: Analysis and implementation" (https://web.archive.org/web/20091007160354/http://www.cs.umd.edu/~mount/Papers/pami02.pdf) (PDF). *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **24** (7): 881–892. Bibcode:2002ITPAM..24..881K (https://ui.adsabs.harvard.edu/abs/2002ITPAM..24..881K). doi:10.1109/TPAMI.2002.1017616 (https://doi.org/10.1109%2FTPAMI.2002.1017616). S2CID 12003435 (https://api.semanticscholar.org/CorpusID:12003435). Archived from the original (http://www.cs.umd.edu/~mount/Papers/pami02.pdf) (PDF) on 2009-10-07. Retrieved 2009-04-24.
38. Drake, Jonathan (2012). "Accelerated *k*-means with adaptive distance bounds" (http://opt.kyb.tuebingen.mpg.de/papers/opt2012_paper_13.pdf) (PDF). *The 5th NIPS Workshop on Optimization for Machine Learning, OPT2012*.

39. Dhillon, I. S.; Modha, D. M. (2001). "Concept decompositions for large sparse text data using clustering" (https://doi.org/10.1023%2Fa%3A1007612920971). *Machine Learning*. **42** (1): 143–175. Bibcode:2001MLear..42..143D (https://ui.adsabs.harvard.edu/abs/2001MLear..42..143D). doi:10.1023/a:1007612920971 (https://doi.org/10.1023%2Fa%3A1007612920971).
40. Steinbach, M.; Karypis, G.; Kumar, V. (2000). " "A comparison of document clustering techniques". In". *KDD Workshop on Text Mining*. **400** (1): 525–526.
41. Pelleg, D.; & Moore, A. W. (2000, June). "X-means: Extending *k*-means with Efficient Estimation of the Number of Clusters (http://cs.uef.fi/~zhao/Courses/Clustering2012/Xmeans.pdf) Archived (https://web.archive.org/web/20160909085805/http://cs.uef.fi/~zhao/Courses/Clustering2012/Xmeans.pdf) 2016-09-09 at the Wayback Machine". In *ICML*, Vol. 1
42. Hamerly, Greg; Elkan, Charles (2004). "Learning the k in k-means" (http://papers.nips.cc/paper/2526-learning-the-k-in-k-means.pdf) (PDF). *Advances in Neural Information Processing Systems*. **16**: 281.
43. Amorim, R. C.; Mirkin, B. (2012). "Minkowski Metric, Feature Weighting and Anomalous Cluster Initialisation in *k*-Means Clustering". *Pattern Recognition*. **45** (3): 1061–1075. doi:10.1016/j.patcog.2011.08.012 (https://doi.org/10.1016%2Fj.patcog.2011.08.012).
44. Amorim, R. C.; Hennig, C. (2015). "Recovering the number of clusters in data sets with noise features using feature rescaling factors". *Information Sciences*. **324**: 126–145. arXiv:1602.06989 (https://arxiv.org/abs/1602.06989). doi:10.1016/j.ins.2015.06.039 (https://doi.org/10.1016%2Fj.ins.2015.06.039). S2CID 315803 (https://api.semanticscholar.org/CorpusID:315803).
45. Sculley, David (2010). "Web-scale *k*-means clustering" (http://dl.acm.org/citation.cfm?id=1772862). *Proceedings of the 19th international conference on World Wide Web*. ACM. pp. 1177–1178. Retrieved 2016-12-21.
46. Telgarsky, Matus. "Hartigan's Method: *k*-means Clustering without Voronoi" (http://proceedings.mlr.press/v9/telgarsky10a/telgarsky10a.pdf) (PDF).
47. Piccialli, Veronica; Sudoso, Antonio M.; Wiegele, Angelika (2022-03-28). "SOS-SDP: An Exact Solver for Minimum Sum-of-Squares Clustering" (http://pubsonline.informs.org/doi/10.1287/ijoc.2022.1166). *INFORMS Journal on Computing*. **34** (4): 2144–2162. arXiv:2104.11542 (https://arxiv.org/abs/2104.11542). doi:10.1287/ijoc.2022.1166 (https://doi.org/10.1287%2Fijoc.2022.1166). ISSN 1091-9856 (https://search.worldcat.org/issn/1091-9856). S2CID 233388043 (https://api.semanticscholar.org/CorpusID:233388043).
48. Bagirov, A. M.; Taheri, S.; Ugon, J. (2016). "Nonsmooth DC programming approach to the minimum sum-of-squares clustering problems". *Pattern Recognition*. **53**: 12–24. Bibcode:2016PatRe..53...12B (https://ui.adsabs.harvard.edu/abs/2016PatRe..53...12B). doi:10.1016/j.patcog.2015.11.011 (https://doi.org/10.1016%2Fj.patcog.2015.11.011).
49. Fränti, Pasi (2018). "Efficiency of random swap clustering" (https://doi.org/10.1186%2Fs40537-018-0122-y). *Journal of Big Data*. **5** (1) 13: 1–21. doi:10.1186/s40537-018-0122-y (https://doi.org/10.1186%2Fs40537-018-0122-y).
50. Hansen, P.; Mladenovic, N. (2001). "J-Means: A new local search heuristic for minimum sum of squares clustering". *Pattern Recognition*. **34** (2): 405–413. Bibcode:2001PatRe..34..405H (https://ui.adsabs.harvard.edu/abs/2001PatRe..34..405H). doi:10.1016/S0031-3203(99)00216-2 (https://doi.org/10.1016%2FS0031-3203%2899%2900216-2).
51. Krishna, K.; Murty, M. N. (1999). "Genetic k-means algorithm" (https://www.researchgate.net/publication/5600582). *IEEE Transactions on Systems, Man, and Cybernetics - Part B: Cybernetics*. **29** (3): 433–439. Bibcode:1999ITSMB..29..433K (https://ui.adsabs.harvard.edu/abs/1999ITSMB..29..433K). doi:10.1109/3477.764879 (https://doi.org/10.1109%2F3477.764879). PMID 18252317 (https://pubmed.ncbi.nlm.nih.gov/18252317).

52. Gribel, Daniel; Vidal, Thibaut (2019). "HG-means: A scalable hybrid metaheuristic for minimum sum-of-squares clustering". *Pattern Recognition*. **88**: 569–583. arXiv:1804.09813 (https://arxiv.org/abs/1804.09813). doi:10.1016/j.patcog.2018.12.022 (https://doi.org/10.1016%2Fj.patcog.2018.12.022). S2CID 13746584 (https://api.semanticscholar.org/CorpusID:13746584).
53. Mirkes, E. M. "K-means and *k*-medoids applet" (http://www.math.le.ac.uk/people/ag153/homepage/KmeansKmedoids/Kmeans_Kmedoids.html). Retrieved 2 January 2016.
54. Kulis, Brian; Jordan, Michael I. (2012-06-26). "Revisiting *k*-means: new algorithms via Bayesian nonparametrics" (https://icml.cc/2012/papers/291.pdf) (PDF). *ICML*. Association for Computing Machinery. pp. 1131–1138. ISBN 978-1-4503-1285-1.
55. Beagrie, Robert A.; Scialdone, Antonio; Schueler, Michael; Kraemer, Daniel C. A.; Chotalia, Malcolm; Xie, Songtao Q.; Barbieri, Marco; de Santiago, Ignacio; Lavitas, Laura-Maria; Fraser, James; Dostie, Josée; Pombo, Ana (2017). "Complex multi-enhancer contacts captured by genome architecture mapping" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5366070). *Nature*. **543** (7646): 519–524. Bibcode:2017Natur.543..519B (https://ui.adsabs.harvard.edu/abs/2017Natur.543..519B). doi:10.1038/nature21411 (https://doi.org/10.1038%2Fnature21411). PMC 5366070 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5366070). PMID 28273065 (https://pubmed.ncbi.nlm.nih.gov/28273065).
56. Eisen, Michael B.; Spellman, Paul T.; Brown, Patrick O.; Botstein, David (1998). "Cluster analysis and display of genome-wide expression patterns" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC24541). *Proceedings of the National Academy of Sciences*. **95** (25): 14863–14868. Bibcode:1998PNAS...9514863E (https://ui.adsabs.harvard.edu/abs/1998PNAS...9514863E). doi:10.1073/pnas.95.25.14863 (https://doi.org/10.1073%2Fpnas.95.25.14863). PMC 24541 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC24541). PMID 9843981 (https://pubmed.ncbi.nlm.nih.gov/9843981).
57. Coates, Adam; Ng, Andrew Y. (2012). "Learning feature representations with *k*-means" (https://cs.stanford.edu/~acoates/papers/coatesng_nntot2012.pdf) (PDF). In Montavon, G.; Orr, G. B.; Müller, K.-R. (eds.). *Neural Networks: Tricks of the Trade*. Springer.
58. Csurka, Gabriella; Dance, Christopher C.; Fan, Lixin; Willamowski, Jutta; Bray, Cédric (2004). *Visual categorization with bags of keypoints* (https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/csurka-eccv-04.pdf) (PDF). ECCV Workshop on Statistical Learning in Computer Vision.
59. Coates, Adam; Lee, Honglak; Ng, Andrew Y. (2011). *An analysis of single-layer networks in unsupervised feature learning* (https://web.archive.org/web/20130510120705/http://www.stanford.edu/~acoates/papers/coatesleeng_aistats_2011.pdf) (PDF). International Conference on Artificial Intelligence and Statistics (AISTATS). Archived from the original (http://www.stanford.edu/~acoates/papers/coatesleeng_aistats_2011.pdf) (PDF) on 2013-05-10.
60. Schwenker, Friedhelm; Kestler, Hans A.; Palm, Günther (2001). "Three learning phases for radial-basis-function networks". *Neural Networks*. **14** (4–5): 439–458. Bibcode:2001NN.....14..439S (https://ui.adsabs.harvard.edu/abs/2001NN.....14..439S). doi:10.1016/s0893-6080(01)00027-2 (https://doi.org/10.1016%2Fs0893-6080%2801%2900027-2). PMID 11411631 (https://pubmed.ncbi.nlm.nih.gov/11411631).
61. Lin, Dekang; Wu, Xiaoyun (2009). *Phrase clustering for discriminative learning* (http://www.aclweb.org/anthology/P/P09/P09-1116.pdf) (PDF). Annual Meeting of the ACL and IJCNLP. pp. 1030–1038.
62. "Parallelization of the K-Means Algorithm with Applications to Big Data Clustering" (https://arxiv.org/html/2405.12052). *arxiv.org*. Retrieved 2026-04-21.
63. García Pérez, A. E., et al. (2019). Machine learning in APOGEE: Identification of stellar populations through chemical composition. *Astronomy & Astrophysics*. https://www.aanda.org/articles/aa/full_html/2019/09/aa35223-19/aa35223-19.html

64. Slijepcevic, I. V., et al. (2025). Semi-supervised classification of stars, galaxies and quasars using K-means and random-forest approaches. *Astronomy & Astrophysics*. https://www.aanda.org/articles/aa/full_html/2025/08/aa55620-25/aa55620-25.html
65. Arthur, David; Vassilvitskii, Sergei. "K-means++: The Advantages of Careful Seeding" (https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf) (PDF). *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*: 1027–1035.
66. Bahmani, Bahman; Moseley, Benjamin; Vattani, Andrea; Kumar, Ravi; Vassilvitskii, Sergei (2012). "Scalable K-means++" (https://theory.stanford.edu/~sergei/papers/vldb12-kmpar.pdf) (PDF). *Proceedings of the VLDB Endowment*. **5** (7): 622–633. doi:10.14778/2180912.2180915 (https://doi.org/10.14778%2F2180912.2180915).
67. Press, W. H.; Teukolsky, S. A.; Vetterling, W. T.; Flannery, B. P. (2007). "Section 16.1. Gaussian Mixture Models and *k*-Means Clustering" (http://apps.nrbook.com/empanel/index.html#pg=842). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). New York (NY): Cambridge University Press. ISBN 978-0-521-88068-8.
68. Kevin P. Murphy (2012). *Machine learning: a probabilistic perspective*. Cambridge, Mass.: MIT Press. ISBN 978-0-262-30524-2. OCLC 810414751 (https://search.worldcat.org/oclc/810414751).
69. Aharon, Michal; Elad, Michael; Bruckstein, Alfred (2006). "K-SVD: An Algorithm for Designing Overcomplete Dictionaries for Sparse Representation" (https://web.archive.org/web/20191202222055/http://www.cs.technion.ac.il/FREDDY/papers/120.pdf) (PDF). *IEEE Transactions on Signal Processing*. **54** (11): 4311. Bibcode:2006ITSP...54.4311A (https://ui.adsabs.harvard.edu/abs/2006ITSP...54.4311A). doi:10.1109/TSP.2006.881199 (https://doi.org/10.1109%2FTSP.2006.881199). S2CID 7477309 (https://api.semanticscholar.org/CorpusID:7477309). Archived from the original (http://www.cs.technion.ac.il/FREDDY/papers/120.pdf) (PDF) on 2019-12-02. Retrieved 2019-12-02.
70. Zha, Hongyuan; Ding, Chris; Gu, Ming; He, Xiaofeng; Simon, Horst D. (December 2001). "Spectral Relaxation for *k*-means Clustering" (http://ranger.uta.edu/~chqding/papers/Zha-Kmeans.pdf) (PDF). *Neural Information Processing Systems Vol.14 (NIPS 2001)*: 1057–1064.
71. Ding, Chris; He, Xiaofeng (July 2004). "K-means Clustering via Principal Component Analysis" (http://ranger.uta.edu/~chqding/papers/KmeansPCA1.pdf) (PDF). *Proceedings of International Conference on Machine Learning (ICML 2004)*: 225–232.
72. Drineas, Petros; Frieze, Alan M.; Kannan, Ravi; Vempala, Santosh; Vinay, Vishwanathan (2004). "Clustering large graphs via the singular value decomposition" (http://www.cc.gatech.edu/~vempala/papers/dfkvv.pdf) (PDF). *Machine Learning*. **56** (1–3): 9–33. Bibcode:2004MLear..56....9D (https://ui.adsabs.harvard.edu/abs/2004MLear..56....9D). doi:10.1023/b:mach.0000033113.59016.96 (https://doi.org/10.1023%2Fb%3Amach.0000033113.59016.96). S2CID 5892850 (https://api.semanticscholar.org/CorpusID:5892850). Retrieved 2012-08-02.
73. Cohen, Michael B.; Elder, Sam; Musco, Cameron; Musco, Christopher; Persu, Madalina (2014). "Dimensionality reduction for *k*-means clustering and low rank approximation (Appendix B)". arXiv:1410.6801 (https://arxiv.org/abs/1410.6801) [cs.DS (https://arxiv.org/archive/cs.DS)].
74. Little, Max A.; Jones, Nick S. (2011). "Generalized methods and solvers for noise removal from piecewise constant signals. I. Background theory" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3191861). *Proceedings of the Royal Society A*. **467** (2135): 3088–3114. Bibcode:2011RSPSA.467.3088L (https://ui.adsabs.harvard.edu/abs/2011RSPSA.467.3088L). doi:10.1098/rspa.2010.0671 (https://doi.org/10.1098%2Frspa.2010.0671). PMC 3191861 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3191861). PMID 22003312 (https://pubmed.ncbi.nlm.nih.gov/22003312).
75. Vinnikov, Alon; Shalev-Shwartz, Shai (2014). "K-means Recovers ICA Filters when Independent Components are Sparse" (http://www.cs.huji.ac.il/~shais/papers/KmeansICA_ICML2014.pdf) (PDF). *Proceedings of the International Conference on Machine Learning (ICML 2014)*.

Retrieved from "https://en.wikipedia.org/w/index.php?title=K-means_clustering&oldid=1380990513"